

CS6.201: Introduction to Software Systems

Python Session - 2

`numpy` and `matplotlib`

Prakhar Jain, Amey Karan, Neel Amrutia, Vivek Hruday,
Chirag Dhamija

March 5, 2025

Laptops Off!

Life Skills!

```
<[REDACTED]_ca1>.pdf
```

Apparently, following simple instructions is harder than rocket science.

Real-Time Operating System

This is the answer for whoever asked

Embedded systems almost always run **real-time operating systems**. A real-time system is used when rigid time requirements have been placed on the operation of a processor or the flow of data; thus, it is often used as a control device in a dedicated application. Sensors bring data to the computer. The computer must analyze the data and possibly adjust controls to modify the sensor inputs. Systems that control scientific experiments, medical imaging systems, industrial control systems, and certain display systems are real-time systems. Some automobile-engine fuel-injection systems, home-appliance controllers, and weapon systems are also real-time systems.

A real-time system has well-defined, fixed time constraints. Processing *must* be done within the defined constraints, or the system will fail. For instance, it would not do for a robot arm to be instructed to halt *after* it had smashed into the car it was building. A real-time system functions correctly only if it returns the correct result within its time constraints. Contrast this system with a traditional laptop system where it is desirable (but not mandatory) to respond quickly.

Examples of Real-Time Operating Systems include Wind River VxWorks, QNX, FreeRTOS, RTLinux, etc.

Abraham Silberschatz, Peter Baer Galvin, and Greg Gagne. *Operating System Concepts, 10th Edition*. Wiley, 2018. ISBN: 978-1-118-06333-0. URL: <http://os-book.com/OS10/index.html>

numpy

**Pronouncing numpy
num-pie num-pee**



**Which side are
you on?**

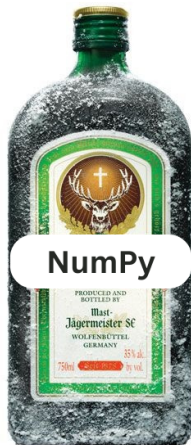
Introduction to Numpy

- Fundamental package for scientific computing with Python.
- Features an N-dimensional array object.
- Provides tools for linear algebra, Fourier transforms, random number capabilities.
- Serves as a building block for other packages (e.g., SciPy, Pandas).
- Works well with plotting libraries (matplotlib, seaborn, plotly etc.).
- Open source.
- blah, blah, blah...

Why Use NumPy over Python Objects?

- NumPy is implemented in C, making it much faster than native Python lists.
- Uses **contiguous memory allocation** which improves cache performance.
- Supports **vectorized operations** (*covered later*), eliminating the need for loops.
- Performs **hardware-level optimizations** using optimized BLAS and LAPACK libraries.
- More memory-efficient compared to Python lists and tuples.
- Essential for high-performance scientific computing.

**How
Much
Do
You
Know
About ...**



Basics

```
1  import numpy as np
2
3  A = np.array([[1, 2, 3],
4                [4, 5, 6]])
5
6  print(A)
7  # Output:
8  # [[1 2 3]
9  #   [4 5 6]]
10
11 Af = np.array([1, 2, 3], dtype=float)
12 # Output:
13 # [1. 2. 3.]
14
```

Array Creation and Initialization

```
1 np.arange(0, 1, 0.2)
2 # array([0. , 0.2, 0.4, 0.6, 0.8])
3
4 np.linspace(0, 2*np.pi, 4)
5 # array([0. , 2.09, 4.18, 6.28])
6
7 A = np.zeros((2, 3))
8 # array([[0., 0., 0.],
9 #        [0., 0., 0.]])
10 # np.ones, np.diag
11
12 A.shape
13 # (2, 3)
```

Numpy arrays are mutable

```
1 A = np.zeros((2, 2))
2 # array([[ 0.,  0.],
3 #        [ 0.,  0.]])
4
5 C = A
6 C[0, 0] = 1
7
8 print(A)
9 # [[ 1.  0.]
10 #   [ 0.  0.]]
11
```

You can prevent this behavior by using the `np.copy()` function to create an independent copy of the array.

Array Attributes

```
1 a = np.arange(10).reshape((2, 5))
2 print(a.ndim)    # 2 (dimensions)
3 print(a.shape)   # (2, 5)
4 print(a.size)    # 10 (number of elements)
5 print(a.T)       # Transpose
6 a.dtype          # Data type
```

Basic Operations

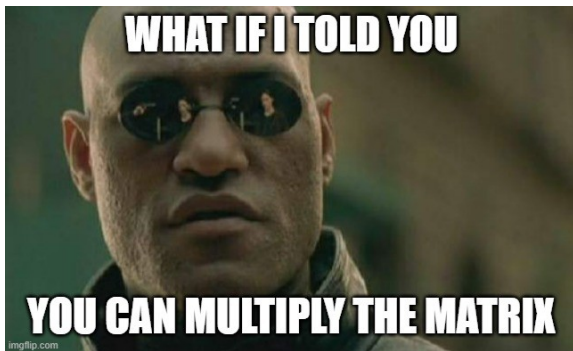
```
1 a = np.arange(4) # array([0, 1, 2, 3])
2
3 b = np.array([2, 3, 2, 4])
4 print(a * b)      # array([0, 3, 4, 12])
5
6 print(b - a)      # array([2, 2, 0, 1])
7 c = [2, 3, 4, 5]
8 print(a * c)      # array([0, 3, 8, 15])
```

Vector Operations

```
1  # Example vectors
2  u = [1, 2, 3]
3  v = [1, 1, 1]
4
5  # Inner Product
6  np.inner(u, v) # Output: 6
7
8  # Outer Product
9  np.outer(u, v) # Output:
10 #             array([[1, 1, 1],
11 #                    [2, 2, 2],
12 #                    [3, 3, 3]])
13
14 # Dot Product (Matrix Multiplication)
15 np.dot(u, v) # Output: 6
```

Matrix Operations

- `np.add(A, B)` : Adds matrices A and B element-wise.
- `np.subtract(A, B)` : Subtracts matrix B from matrix A element-wise.
- `np.multiply(A, B)` : Multiplies matrices A and B element-wise.
- `np.divide(A, B)` : Divides matrix A by matrix B element-wise.
- `np.dot(A, B)` : Computes the dot product of matrices A and B .



Matrix Operations

- `np.transpose(A)`: Transposes matrix A (swaps rows and columns).
- `np.matmul(A, B)`: Performs matrix multiplication (supports 2D arrays). Performs the same functions as `np.dot`
- `np.identity(n)`: Creates an $n \times n$ identity matrix.
- `np.trace(A)`: Computes the trace of matrix A (sum of diagonal elements).

Matrix Operations

```
1 # Define matrices
2 A = np.ones((3, 2))
3 # array([[ 1.,  1.],
4 #        [ 1.,  1.],
5 #        [ 1.,  1.]])
6 A.T
7 # array([[ 1.,  1.,  1.],
8 #        [ 1.,  1.,  1.]])
9
10 B = np.ones((2, 3))
11 # array([[ 1.,  1.,  1.],
12 #        [ 1.,  1.,  1.]])
```

Matrix Operations

```
1 # Matrix operations
2
3 np.dot(B.T, A.T)
4 # array([[ 2.,  2.,  2.],
5 #        [ 2.,  2.,  2.],
6 #        [ 2.,  2.,  2.]])
7
8 np.dot(A, B.T)
9 # Error: ValueError: shapes (3,2) and (3,2) not aligned: ...
10 # ... 2 (dim 1) != 3 (dim 0)
```

1-D Array Slicing Example

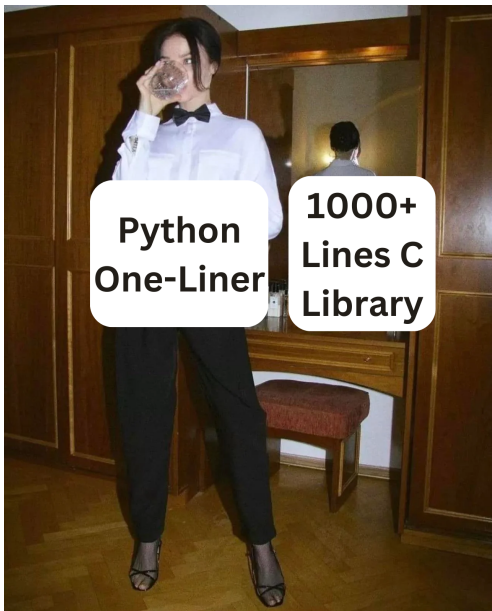
```
1 # Generate a 1-D array
2 a = np.array([0.25, 0.56, 0.98, 0.13, 0.72])
3
4 ## Select elements from index 2 to the end
5 a[2:] # array([ 0.98,  0.13,  0.72])
6
7 ## Select elements from index 1 to 4 (exclusive)
8 a[1:4] # array([ 0.56,  0.98,  0.13])
9
10 # Select every second element
11 a[::2] # array([ 0.25,  0.98,  0.72])
```

1-D Array Slicing Example (Continued)

```
12 # Select elements in reverse order
13 a[::-1] # array([ 0.72,  0.13,  0.98,  0.56,  0.25])
14
15 # Select elements with a negative step from index 4 to 0
   ↳ (exclusive)
16 a[4:0:-1] # array([0.13,  0.98,  0.56,  0.25])
17
18 # Select the last element (negative indexing)
19 a[-1] # 0.72
20
21 # Select the second to last element (negative indexing)
22 a[-2]
23 # Output: 0.13
```

2-D Array Slicing

```
1  a = np.array([[ 0.25,  0.56,  0.98,  0.13,  0.72],
2                [ 0.43,  0.15,  0.67,  0.89,  0.24],
3                [ 0.91,  0.78,  0.64,  0.38,  0.55],
4                [ 0.19,  0.82,  0.13,  0.29,  0.71]])
5
6  # Select the third row, all columns
7  a[2, :]
8  # Output: array([ 0.91,  0.78,  0.64,  0.38,  0.55])
9
10 # Select the 2nd and 3rd rows, all columns
11 a[1:3]
12 # Output: array([[ 0.43,  0.15,  0.67,  0.89,  0.24],
13                [ 0.91,  0.78,  0.64,  0.38,  0.55]])
```



2-D Array Slicing (Continued)

```
14 # Select rows 1 through 3 (exclusive of 3), columns 1 through 4
    ↪ (exclusive of 4)
15 a[1:3, 1:4]
16 # Output: array([[ 0.15,  0.67,  0.89],
17 #               [ 0.78,  0.64,  0.38]])
18 # Select rows in reverse order, every other column
19 a[::-1, ::2]
20 # Output: array([[ 0.19,  0.13,  0.71],
21 #               [ 0.91,  0.64,  0.55],
22 #               [ 0.43,  0.67,  0.24],
23 #               [ 0.25,  0.98,  0.72]])
24
25 # Select the last element from the last row (negative indexing)
26 a[-1, -1]
27 # Output: 0.71
```

Reshaping Arrays

```
1 # Create a 1D array of size 12
2 a = np.arange(12)
3 print("Original array:")
4 print(a)
5 # Original array:
6 # [ 0  1  2  3  4  5  6  7  8  9 10 11]
7
8 # Reshape the array to 3x4
9 reshaped = a.reshape(3, 4)
10 print("\nReshaped array (3x4):")
11 print(reshaped)
12
13 # Reshaped array (3x4):
14 # [[ 0  1  2  3]
15 #   [ 4  5  6  7]
16 #   [ 8  9 10 11]]
```

Rules of Reshaping

The total number of elements in the original array must equal the total number of elements in the reshaped array:

$$\text{Original Size} = \prod (\text{original dimensions}) = \prod (\text{new dimensions})$$

- Example 1: A 1D array with 12 elements can be reshaped into a 3×4 array because $3 \times 4 = 12$.
- Example 2: Attempting to reshape a 1D array with 12 elements into a 3×5 array raises an error because $3 \times 5 \neq 12$.

```
1 # Reshape Example
2 a = np.arange(12)
3
4 # Valid reshaping
5 reshaped = a.reshape(3, 4)
6
7 # Invalid reshaping (raises an error)
8 invalid = a.reshape(3, 5) # ValueError: cannot reshape array
```

Random Sampling with NumPy

- For random sampling we use `np.random` module.

Random Sampling Functions - 1

- `np.random.rand(d0, d1, ..., dn)`: Generates random values from a uniform distribution in the range $[0, 1)$ with a specified shape. For example, `rand(2, 3)` will generate a 2x3 array of random values.
- `np.random.randn(d0, d1, ..., dn)`: Generates random values from a standard normal distribution (mean=0, variance=1) with the given shape. For example, `randn(2, 3)` will produce a 2x3 array with values from the standard normal distribution.

Random Sampling Functions - 2

- `np.random.randint(lo, hi, size)`: Generates random integers from the range $[lo, hi)$, where `lo` is the lower bound (inclusive) and `hi` is the upper bound (exclusive). The `size` argument specifies the shape of the output array. For example, `randint(0, 10, (2, 3))` will produce a 2x3 array with integers between 0 and 9.
- `np.random.choice(a, size, repl, p)`: Randomly samples elements from array `a`. The `size` specifies the number of elements to sample. If `repl=True`, the same element can be selected multiple times (sampling with replacement), otherwise, each element can only be selected once (sampling without replacement). The `p` parameter specifies the probability distribution for the sampling.

Understanding Seeds in Random Sampling

- In random sampling, a **seed** is an initial value used by a pseudorandom number generator (PRNG) to produce a sequence of random numbers.
- The seed ensures that random sampling can be reproduced. If you use the same seed value, you will get the same random numbers each time, which is useful for debugging and reproducibility in experiments.

Setting a Seed in NumPy

`np.random.seed(seed)`: Sets the seed for the random number generator. Once the seed is set, all subsequent random number generation will be deterministic and based on that seed.

```
1 # Set the seed for reproducibility
2 np.random.seed(42)
3 print(np.random.rand(2,3))
4 # array([[0.37454012, 0.95071431, 0.73199394],
5 #        [0.59865848, 0.15601864, 0.15599452]])
6
7 # seed again
8 np.random.seed(42)
9 print(np.random.rand(2,3))
10 # array([[0.37454012, 0.95071431, 0.73199394],
11 #        [0.59865848, 0.15601864, 0.15599452]])
12
13 # Generates the same numbers every time after setting the seed
```


Introduction to Masking in NumPy

- Masking allows for filtering elements of an array based on specified conditions.
- The result is an array where only the elements satisfying the condition(s) are included.
- Masking uses Boolean expressions to create a mask (True/False values) that is applied to the array.
- Common logical operators used in masking:
 - ▶ `&` (AND) – for multiple conditions that must be true
 - ▶ `|` (OR) – for conditions where at least one must be true

Example 1: Using & (AND) and — (OR)

- Masking with AND condition: values greater than 2 AND less than 5

```
1 arr = np.array([1, 2, 3, 4, 5, 6])
2
3 # Masking with AND condition
4 mask = (arr > 2) & (arr < 5)
5 masked_array = arr[mask]
6 print(masked_array) # Output: [3 4]
```

- Masking with OR condition: values less than 2 OR greater than 5

```
1 mask = (arr < 2) | (arr > 5)
2 masked_array = arr[mask]
3 print(masked_array) # Output: [1 6]
```

Example 2: Using & (AND) for Multiple Conditions

- Masking with AND condition: values greater than 2 AND even numbers

```
1  arr = np.array([1, 2, 3, 4, 5, 6])
2
3  # Masking with AND condition: greater than 2 AND even
   ↪ numbers
4  mask = (arr > 2) & (arr % 2 == 0)
5  masked_array = arr[mask]
6  print(masked_array)  # Output: [4 6]
```

Important NumPy Functions

- `np.concatenate()` : Joins two or more arrays along an existing axis.
- `np.mean()` : Computes the mean (average) of array elements.
- `np.median()` : Computes the median of array elements.
- `np.std()` : Computes the standard deviation of array elements.
- `np.unique()` : Finds the unique elements of an array.
- `np.split()` : Splits an array into multiple sub-arrays.
- `np.argmax()` : Returns the indices of the maximum values along an axis.
- `np.argmin()` : Returns the indices of the minimum values along an axis.



Important NumPy Functions

- `np.argsort()` : Returns the indices that would sort an array.
- `np.hstack()` : Stacks arrays in sequence horizontally (column-wise).
- `np.vstack()` : Stacks arrays in sequence vertically (row-wise).
- `np.repeat()` : Repeats elements of an array.
- `np.isnan()` : Tests element-wise for NaNs (Not a Number).
- `np.isin()` : Tests whether elements of an array are in another array.
- `np.newaxis` : Adds a new axis to an array, used for reshaping or increasing dimensions.



It doesn't end here...

NumPy's submodules:

- `np.linalg`: Linear algebra functions, such as matrix decompositions, solving linear systems, eigenvalues/eigenvectors, and more.
- `np.random`: Provides a suite of functions for generating random numbers, including probability distributions and random sampling.
- `np.fft`: Fast Fourier Transform functions for signal processing, spectral analysis, and efficient computation of discrete Fourier transforms.
- and more...

<https://numpy.org/doc/stable/reference/index.html>

SIMD

- **SIMD** stands for **Single Instruction Multiple Data**.
- Scalar operations process one data point at a time, whereas **SIMD** operations allow a single instruction to operate on multiple data points simultaneously.
- Commonly used in multimedia, graphics, and scientific computing.
- Modern CPUs and GPUs leverage **SIMD** for faster parallel computations.

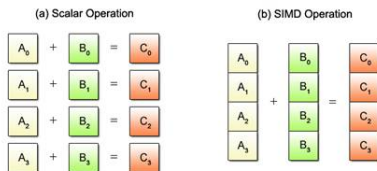


Figure: Scalar vs. SIMD Operations

Restrictions of SIMD (Self-Study)

- SIMD only works on uniform operations.
- Cannot process different operations on different data elements simultaneously.
- Data must be organized in a vectorized format.

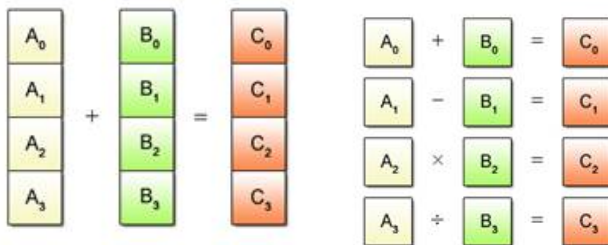


Figure: Processable (Left) vs Unprocessable (Right) SIMD Patterns

Advantages of SIMD (Self-Study)

- Faster computations for large datasets.
- Efficient for repetitive tasks like image and audio processing.
- Reduces the number of instructions executed.
- Boosts performance in machine learning algorithms.

Vectorization

- Vectorization refers to replacing **explicit loops** with optimized, low-level operations.
- Vectorization makes use of **SIMD** to perform multiple operations simultaneously.
- Eliminates the need for loops in high-level languages like Python.

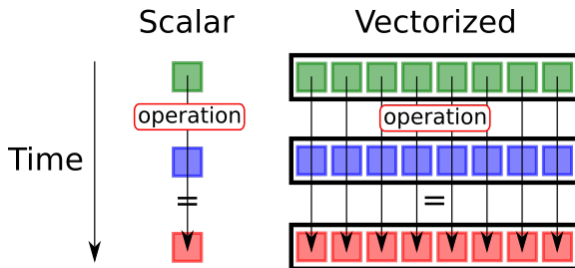


Figure: Vectorization Workflow

Advantages of Vectorization (Self-Study)

- Improves speed and memory efficiency.
- Especially useful in deep learning for faster model training on large datasets.
- Reduces the need for explicit loops, making code more concise and readable.
- Utilizes optimized low-level implementations like BLAS and SIMD instructions.
- Enables parallel execution on modern hardware such as GPUs and multi-core CPUs.
- Minimizes overhead from control flow statements.

Vectorization vs For-Loops

Example: Dot Product

```
1 import numpy as np
2 from time import time
3
4 NO_OF_ELEMENTS = int(1e6)
5 w = np.random.rand(NO_OF_ELEMENTS)
6 x = np.random.rand(NO_OF_ELEMENTS)
7 c = np.zeros(NO_OF_ELEMENTS)
8
9 start_time = time()
10 for i in range(NO_OF_ELEMENTS): # Non-vectorized implementation
11     c += w[i] * x[i]
12 c = np.dot(w, x) # Vectorized Implementation
```

- Vectorized version is **faster** due to hardware-level optimizations.
- Most NumPy functions support vectorized operations.

SIMD, Vectorization, and Broadcasting

Optimizing Performance in NumPy:

- **SIMD** (Single Instruction, Multiple Data): A technique for processing multiple data points with a single instruction, enabling parallel processing for better performance.
- **Vectorization**: Leveraging NumPy's ability to perform operations on entire arrays without explicit loops, allowing for faster and more efficient computations.
- **Broadcasting**: A powerful feature that enables NumPy to perform operations on arrays of different shapes without needing to reshape them, promoting memory efficiency.

Explore Broadcasting to get a better understanding of how NumPy works!

Practice, Practice, Practice...



Summary

- Numpy provides a powerful toolkit for numerical computation.
- Offers support for arrays, linear algebra, Fourier transforms, and random sampling.
- Highly optimized and widely used in scientific computing.
- **Next Steps:**
 - ▶ Explore pandas for data manipulation.
 - ▶ Learn SciPy for advanced scientific computing.
 - ▶ Use scikit-learn for machine learning.
 - ▶ Try XGBoost for gradient boosting.
 - ▶ PyTorch for deep learning (Learning PyTorch will prepare you for most of the ML/DL courses here).
- **Side Project:** Implement a neural network from scratch using NumPy to deepen your understanding. You'll need an understanding of how backpropagation works.

Questions?

matplotlib

Matplotlib is a powerful Python library used for creating static, animated, and interactive visualizations. It is widely used for data visualization in scientific computing.

Installation

To install Matplotlib, use the following command:

```
1 pip install matplotlib
```

Basic Plot

A simple line plot can be created as follows:

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 x = [1, 2, 3, 4, 5]
5 y = [10, 20, 25, 30, 40]
6
7 plt.plot(x, y, marker='o', linestyle='-', color='b')
8 plt.xlabel('X Axis')
9 plt.ylabel('Y Axis')
10 plt.title('Basic Line Plot')
11 plt.show()
```

Scatter Plot

A scatter plot can be created using:

```
1 x = np.random.rand(50)
2 y = np.random.rand(50)
3 colors = np.random.rand(50)
4
5 plt.scatter(x, y, c=colors, alpha=0.5, cmap='viridis')
6 plt.colorbar()
7 plt.title('Scatter Plot')
8 plt.show()
```

Scatter Plot and Line Plot on the Same Axes

To combine a scatter plot and a line plot in the same figure, you can use both `plot()` and `scatter()` functions:

```
1  x = np.linspace(0, 10, 100)
2  y = np.sin(x)
3  x_scatter = np.random.rand(30) * 10
4  y_scatter = np.sin(x_scatter) + np.random.randn(30) * 0.1
5
6  plt.plot(x, y, label='Line Plot', color='b') # Line plot
7  plt.scatter(x_scatter, y_scatter, label='Scatter Plot',
8  ↪ color='r') # Scatter plot
9
10 plt.xlabel('X Axis')
11 plt.ylabel('Y Axis')
12 plt.title('Scatter Plot and Line Plot')
13 plt.legend()
14 plt.show()
```


Bar Plot

Creating a bar chart:

```
1 categories = ['A', 'B', 'C', 'D']
2 values = [3, 7, 1, 8]
3
4 plt.bar(categories, values, color=['red', 'blue', 'green',
   ↪ 'purple'])
5 plt.xlabel('Categories')
6 plt.ylabel('Values')
7 plt.title('Bar Chart')
8 plt.show()
```

You can also use `plt.barh()` to create a horizontal chart.

Side by Side Bar Plot

```
1 categories = ['A', 'B', 'C']
2 values1 = [10, 20, 30]
3 values2 = [15, 25, 35]
4
5 barWidth = 0.25
6
7 r1 = np.arange(len(categories)) # Positions for the first set
  ↳ of bars
8 r2 = [x + barWidth for x in r1] # Positions for the second set
  ↳ of bars
9
10 plt.bar(r1, values1, color='b', width=barWidth, label='Series
   ↳ 1')
11 plt.bar(r2, values2, color='r', width=barWidth, label='Series
   ↳ 2')
12
```

Side by Side Bar Plot (Continued)

```
13 # Add labels and title
14 plt.xlabel('Categories')
15 plt.ylabel('Values')
16 plt.title('Side by Side Bar Plot')
17 # Add custom x-axis tick labels
18 plt.xticks([r + barWidth / 2 for r in r1], categories)
19
20 # Add legend
21 plt.legend()
22
23 # Show the plot
24 plt.show()
```

Histogram

A histogram is useful for visualizing data distributions:

```
1 data = np.random.randn(1000)
2
3 plt.hist(data, bins=30, color='blue', edgecolor='black',
4         ↪ alpha=0.7)
5 plt.xlabel('Value')
6 plt.ylabel('Frequency')
7 plt.title('Histogram')
8 plt.show()
```

More plots...

Additionally, `matplotlib` supports other types of plots like:

- Pie Chart
- Heatmap
- Box Plot

Subplots and Figures

Matplotlib provides the `subplot` and `subplots` functions to create multiple plots within a single figure.

Using Subplot

The `subplot` function allows creating multiple plots by specifying the grid layout as `subplot(rows, cols, index)`.

```
1 plt.figure(figsize=(8, 6))
2
3 plt.subplot(2, 1, 1)
4 plt.plot([1, 2, 3], [4, 5, 6], 'r')
5 plt.title('First Subplot')
6
7 plt.subplot(2, 1, 2)
8 plt.plot([1, 2, 3], [10, 20, 30], 'b')
9 plt.title('Second Subplot')
10
11 plt.tight_layout()
12 plt.show()
```

Now let's see some advanced topics and libraries you can dive into to enhance your data visualization skills.

`subplots()` in Matplotlib

Creating multiple plots within a single figure can be achieved with `subplot()` or `subplots()` functions. These allow you to display several visualizations side by side for better comparison.

- `subplot()` allows you to manually define grid layout (e.g., 2 rows and 2 columns) and select the plot's position.
- `subplots()` provides a more flexible approach, returning a figure and an array of axes, ideal for complex layouts.

3D Plotting with Matplotlib

Matplotlib supports 3D plotting via the `mpl_toolkits.mplot3d` module, which is helpful for visualizing relationships in multivariate data.

- Create 3D scatter plots, surface plots, wireframes, etc.
- Ideal for datasets with more than two variables.

Seaborn Integration

Seaborn, built on top of Matplotlib, simplifies the creation of complex visualizations and offers a high-level interface with built-in themes and color palettes.

- Seaborn handles statistical plots like categorical plots and pair plots with minimal code.
- It integrates well with pandas dataframes, automatically handling data aggregation and statistical analysis.

Interactive Plotting with Plotly

Plotly is a library for creating interactive plots, making it ideal for dynamic data exploration in web applications or Jupyter notebooks.

- Supports various charts, including 3D plots, heatmaps, and geographic maps.
- Features like zooming, hovering, and clicking on data points for detailed information enhance data interactivity.

Questions?



Hands On!